Pimpri-Chinchwad Education Trust's
Pimpri-Chinchwad College of Engineering
(An Autonomous Institute)
Affiliated to Savitribai Phule Pune University (SPPU)
ISO 21001:2018 Certified by TUV

Assignment No 2

Name: Soham Kulkarni
PRN: 124B1F056
Subject: Design and Analysis of Algorithms
Batch: A3
GitHub Link: https://github.com/SOHAM0007-CODER/DAA_ASSIGNMNET

1. Topic

Optimize an OTT movie recommendation system by sorting movie records with Quick Sort. Let the user choose rating, release year or popularity. Read records from a CSV file and display the highest-ranked movies for the selected preference.

2. Course Objective

- Understand the computational complexity of algorithms.
- Select a suitable algorithm design strategy for a practical problem.

3. Course Outcome

- Apply Divide and Conquer to sorting practical records.
- Analyze pivot selection, recursion cost and the performance of Quick Sort.

Theory

Quick Sort partitions movies around a copied middle-element pivot and recursively sorts the partitions. The selected key is rating, release year or popularity; the comparator directly orders records descending. The implementation recurses into the smaller side and iterates over the larger side to bound stack depth.

All 4,803 rows from the Kaggle TMDB 5000 Movie Dataset are retained. Rating is vote_average on a 0-10 scale; popularity remains a decimal TMDB score. Missing release year is shown as 0. Quoted CSV titles containing commas are parsed correctly. Quick Sort is not stable.

Working of the Algorithm

- 1. Read all movie records from movies.csv.
- 2. Choose rating, year or popularity; saved input selects rating.
- 3. Partition around the middle pivot and sort both partitions.
- 4. Print every movie, the total count, comparisons, swaps and sorting-only time.

Complexity Analysis

A partition scans its current subarray once. Balanced partitions give about log N recursion levels with O(N) work per level. Highly unbalanced partitions can produce N levels and quadratic work.

Time Complexity

Best Case: O(N log N), when partitions remain balanced.

Average Case: O(N log N) for typical randomly ordered data.

Worst Case: O(N squared), possible with repeated or unfavorable pivot choices.

Space Complexity

O(log N) recursion stack: recurse on the smaller partition and loop over the larger. The vector itself uses O(N) input storage.

Algorithm - Steps

- Step 1: Read the full CSV and the chosen numeric sort key.
- Step 2: Partition directly into descending order and sort the partitions.
- Step 3: Print all 4,803 records and the performance summary.

Pseudocode

```
QUICK_SORT(A,left,right):
    while left < right:
        pivot = copy of middle element
        partition into values >= pivot and <= pivot
        recurse on smaller partition
        continue loop on larger partition
```

Flow of Execution

START -> Load CSV -> Choose key -> Partition and sort -> Print all records -> END.

6. Example

For ratings [3,1,4,2], the final descending order is [4,3,2,1]. The saved full-dataset run sorts by rating and displays all 4,803 movies, including zero-rated rows.

Middle pivots and balanced handling of equal keys help common inputs but do not remove the O(N squared) worst case. Rating ties may appear in any order.

Conclusion

Quick Sort ranks the entire movie dataset by one chosen numeric attribute. The output retains every record and reports sorting statistics. This is attribute-based ranking rather than learned personalization.